

mindx_pay2store: a production-grade mindX service module for autonomous Arweave archival

This deliverable specifies, in production-ready Python 3.12+, a first-class mindX capability module — `mindx_pay2store` — that wraps the existing `pay2store.agenticplace.pythai.net` x402 v2 service into a Soul-Mind-Hands-aligned cognitive tool that BANKON / AgenticPlace agents may invoke autonomously to commit data permanently to Arweave via Turbo. The architecture, data contracts, and code below are intended to be dropped directly into the mindX repository and to begin archiving real agent output in production against Ethereum mainnet, Base mainnet, and Algorand mainnet payment rails.

All files carry the Apache 2.0 license header `(c) 2026 BANKON - all rights reserved`. The file layout is the cypherpunk2048 flat snake_case standard. All async I/O is built on `httpx` and `asyncio`. All data models use Pydantic v2. All logging is structured via `structlog`. All metrics are Prometheus. All retries are tenacity. The circuit breaker is `pybreaker`. Tests are `pytest` + `pytest-asyncio` + `respx` + `testcontainers` + `hypothesis`.

Pinned library matrix, verified May 2026: `httpx==0.28.1`, `pydantic==2.13.4`, `pydantic-settings==2.14.1`, `fastapi==0.113.1`, `uvicorn[standard]==0.46.0`, `structlog==25.5.0`, `nats-py==2.14.0`, `qdrant-client==1.17.1`, `meilisearch-python-sdk==7.0.4`, `kuzu==0.11.3`, `asyncpg==0.31.0`, `prometheus-client==0.25.0`, `opentelemetry-api==1.41.1`, `opentelemetry-instrumentation-httpx==0.62b1`, `tenacity==9.1.4`, `pybreaker==1.4.1`, `pytest==9.0.3`, `pytest-asyncio==1.3.0`, `respx==0.23.1`, `hypothesis==6.151.11`, `testcontainers==4.14.2`, `eth-account==0.13.7`, `py-algorand-sdk==2.8.0`, `web3==7.x`, `typer==0.25.1`, `PyYAML==6.0.3`, `cryptography==48.0.0`, `x402==2.6.0` (pin $\geq 2.6.0$ to avoid GHSA-qr2g-p6q7-w82m). The Turbo SDK at v1.40.2 remains JavaScript-only; the Python service shells out to the `turbo` Node CLI (per the pay2store reference) or talks directly to the Turbo Upload Service over HTTP — `mindx_pay2store` does neither directly because it delegates to the existing pay2store HTTP service. (PyPI + 11)

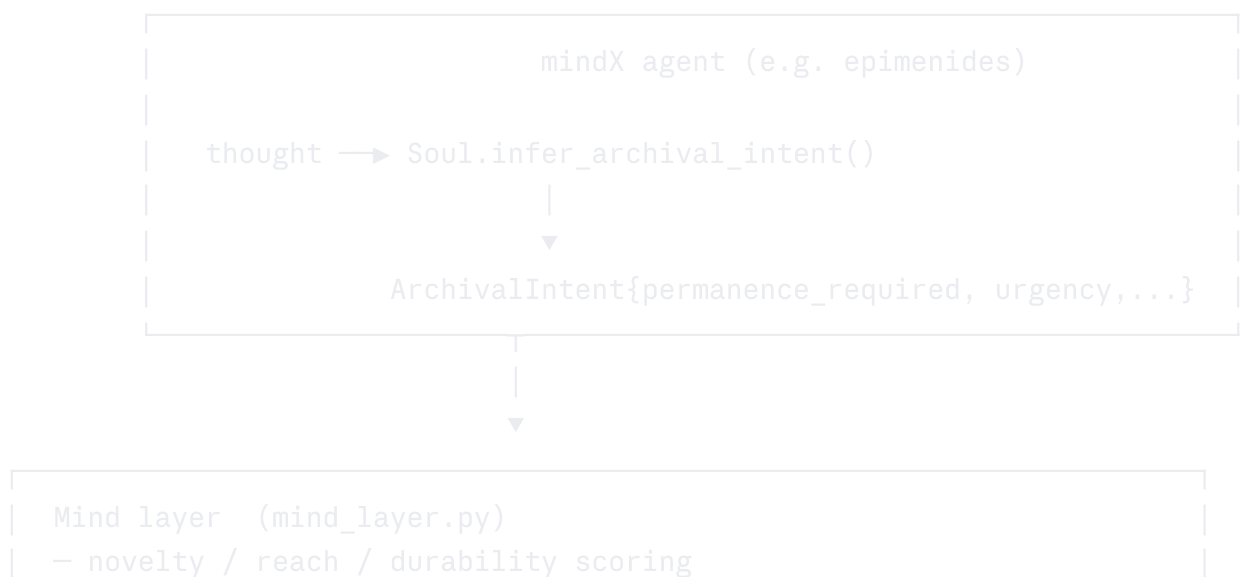
Section 1 — Module architecture

1A. File layout (flat snake_case)

```
mindx_pay2store/
├─ __init__.py
├─ service.py
├─ client.py
├─ policy.py
├─ budget.py
├─ memory_bridge.py
```

```
|— kuzu_index.py
|— qdrant_index.py
|— meilisearch_index.py
|— nats_consumer.py
|— soul_layer.py
|— mind_layer.py
|— hands_layer.py
|— parsec_signer.py
|— verification.py
|— tag_taxonomy.py
|— retry.py
|— metrics.py
|— config.py
|— exceptions.py
|— cli.py
|— api.py
|— policy.example.yaml
|— pyproject.toml
|— Containerfile
|— mindx-pay2store.service
└─ tests/
    |— conftest.py
    |— test_service.py
    |— test_policy.py
    |— test_budget.py
    |— test_memory_bridge.py
    └─ test_integration.py
```

1B. Architectural diagram



```

- PolicyEngine.evaluate(intent, ctx)
- BudgetTracker.reserve(agent_id, est_cost_usd)
→ ArchivalDecision{archive, tier, tags, max_cost_usd}

```

```

| (decision.archive == True)

```

Hands layer (hands_layer.py)

```

Pay2StoreClient (client.py) 402 challenge → pay2store HTTP svc (FastAPI/x402)
                                ←PaymentRequired←

```

```

| sign EIP-3009 / Algorand

```

```

| Turbo upload

```

```

ParsecSigner (parsec_signer.py) ———— PAYMENT-SIGNATURE ———→ Facilitator → Arweave

```

```

| ArchivalReceipt{ar_txid, sha256, cost_usd, ...}

```

VerificationService (verification.py)

```

- arweave.net/tx/{id}/status confirmations
- GraphQL tag match + bundledIn traversal
- BONAFIDE.register(key, ar_txid) on Ethereum
- daily Merkle commit + ENS bankon.eth text record update

```

MindXMemoryBridge (memory_bridge.py)

```

1. NATS JetStream  publish: mindx.memory.archive_complete
2. Kuzu             (:Memory)-[:ARCHIVED_TO]->(:ArweaveTx{txid})
3. Qdrant           upsert(embedding, payload={txid,tags,...})
4. Meilisearch      index(document)
5. Append-only log entry  archive_status: "permanent"

```

The flow is strictly one-way at the cognitive layer (Soul → Mind → Hands), but the memory bridge is fan-out: a single successful archive populates four persistence systems plus the agent's append-only log.

Section 2 — Cognitive layer integration

2A. `soul_layer.py` — high-level archival intent

```
python
```

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Soul layer: high-level intent inference for archival decisions."""
from __future__ import annotations
from dataclasses import dataclass
from enum import StrEnum
from typing import Optional, Protocol
import re
import structlog

log = structlog.get_logger(__name__)

class Urgency(StrEnum):
    IMMEDIATE = "immediate" # archive within seconds
    SOON      = "soon"      # within minutes
    BATCH     = "batch"     # next daily commit
    DEFERRED  = "deferred"  # may be batched into Merkle root only

@dataclass(frozen=True, slots=True)
class ArchivalIntent:
    subject: str
    motivation: str
    urgency: Urgency
    permanence_required: bool
    suggested_tier: str # "cold" | "warm" | "hot"
    confidence: float # 0.0 .. 1.0

class MindXThought(Protocol):
    id: str
    agent_id: str
    content: str
    metadata: dict[str, object]

class MindXIntrospector(Protocol):
    async def reflect(self, prompt: str, *, context: object | None = None) -> str:

_CONSTITUTIONAL_MARKERS = re.compile(
    r"\b(constitution(?:al)?|amendment|covenant|oath|inviolable|"

```

```

    r"governance\s+ruling|DAIO\s+decision)\b", re.I)
_DRAFT_MARKERS = re.compile(
    r"\b(draft|scratch|tentative|wip|working\s+notes|brainstorm)\b", re.I)
_PERMANENCE_MARKERS = re.compile(
    r"\b(forever|permanent(?:ly)?|immutable|on-?chain|archival|"
    r"for\s+the\s+record|let\s+it\s+be\s+known)\b", re.I)

async def infer_archival_intent(
    thought: MindXThought,
    introspector: MindXIntrospector,
) -> Optional[ArchivalIntent]:
    """Decide whether `thought` deserves an archival intent.

    Combines fast heuristic gates with a single mindX self-reflection call.
    Returns None when the thought is clearly ephemeral (e.g. a draft).
    """
    text = thought.content
    if _DRAFT_MARKERS.search(text):
        log.debug("soul.draft_short_circuit", thought_id=thought.id)
        return None

    constitutional = bool(_CONSTITUTIONAL_MARKERS.search(text))
    permanence_signal = bool(_PERMANENCE_MARKERS.search(text)) or constitutional

    prompt = (
        "You are mindX reflecting on whether one of your own thoughts merits "
        "permanent on-chain archival to Arweave. Respond with a JSON object: "
        '{"permanence_required": bool, "urgency": "immediate"|"soon"|"batch"|" '
        '"deferred", "suggested_tier": "cold"|"warm"|"hot", "subject": str, '
        '"motivation": str, "confidence": float}. '
        f"Thought (agent={thought.agent_id}): {text[:4000]}"
    )
    raw = await introspector.reflect(prompt, context=thought.metadata)
    try:
        import json
        data = json.loads(raw)
    except Exception:
        log.warning("soul.reflect_parse_failure", thought_id=thought.id, raw=raw[:200])
        return None

    intent = ArchivalIntent(
        subject=str(data.get("subject", thought.id)),
        motivation=str(data.get("motivation", "")),

```

```

        urgency=Urgency(data.get("urgency", "batch")),
        permanence_required=bool(data.get("permanence_required", False)) or permanence_required,
        suggested_tier=str(data.get("suggested_tier", "cold")),
        confidence=float(data.get("confidence", 0.5)),
    )
    if not intent.permanence_required and intent.confidence < 0.6:
        return None
    log.info("soul.intent_formed", thought_id=thought.id, intent=intent.__dict__)
    return intent

```

The agent manifest declares the capability:

```

yaml

# AgenticPlace agent.yaml
capabilities:
  - tool.mindx.pay2store.intent
    version: "1.0"
    handler: mindx_pay2store.soul_layer:infer_archival_intent

```

2B. `mind_layer.py` — reasoning about archival value

```
python
```

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Mind layer: cost-benefit and policy reasoning over an ArchivalIntent."""
from __future__ import annotations
from dataclasses import dataclass
from typing import Protocol
import structlog
from .soul_layer import ArchivalIntent
from .policy import PolicyEngine, PolicyDecision
from .budget import BudgetTracker

log = structlog.get_logger(__name__)

@dataclass(frozen=True, slots=True)
class MindXContext:
    agent_id: str
    payload_size: int
    content_type: str
    payload_sha256: str
    novelty: float        # 0..1; 1 = unique vs existing memory
    reach: float          # 0..1; expected fan-out to other agents
    importance: float      # 0..1; mindX-assessed importance score

@dataclass(frozen=True, slots=True)
class ArchivalDecision:
    archive: bool
    tier: str              # "cold" | "warm" | "hot"
    tags: dict[str, str]
    max_cost_usd: float
    rationale: str

_TIER_DURABILITY_USD = {"cold": 0.001, "warm": 0.005, "hot": 0.020}

def _stoic_score(intent: ArchivalIntent, ctx: MindXContext) -> float:
    """Composite score in [0,1]; the 'discernment' weight."""
    weights = {"novelty": 0.35, "reach": 0.25, "importance": 0.30, "confidence": 0.1}
    return (weights["novelty"] * ctx.novelty
            + weights["reach"] * ctx.reach
            + weights["importance"] * ctx.importance

```



```
+ weights["confidence"]* intent.confidence)
```

```
def _select_tier(intent: ArchivalIntent, score: float) -> str:
    if intent.permanence_required and score >= 0.85: return "hot"
    if score >= 0.6: return "warm"
    return "cold"
```

```
async def evaluate_archival_decision(
    intent: ArchivalIntent,
    ctx: MindXContext,
    *,
    policy: PolicyEngine,
    budget: BudgetTracker,
) -> ArchivalDecision:
    """Combine philosophical scoring, policy, and budget into a single decision."""
    score = _stoic_score(intent, ctx)
    tier = _select_tier(intent, score)
    est_cost = _TIER_DURABILITY_USD[tier]

    pol: PolicyDecision = await policy.evaluate(
        agent_id=ctx.agent_id,
        size_bytes=ctx.payload_size,
        content_type=ctx.content_type,
        payload_sha256=ctx.payload_sha256,
        importance=ctx.importance,
        tier=tier,
    )
    if not pol.allow:
        return ArchivalDecision(False, tier, {}, 0.0, f"policy_denied: {pol.reason}")

    if not await budget.can_afford(ctx.agent_id, est_cost, tier=tier):
        return ArchivalDecision(False, tier, {}, 0.0, "budget_exhausted")

    tags = {
        "App-Name": "mindX",
        "Agent-Id": ctx.agent_id,
        "Tier": tier,
        "Intent-Subject": intent.subject,
        "Content-Type": ctx.content_type,
        "Payload-Sha256": ctx.payload_sha256,
        **pol.additional_tags,
    }
```

```
log.info("mind.decision", agent_id=ctx.agent_id, score=score, tier=tier, cost=es  
return ArchivalDecision(True, tier, tags, est_cost * 1.25,  
                        f"score={score:.2f} tier={tier} stoic_pass")
```

2C. `hands_layer.py` — execution

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Hands layer: actually performs the pay2store call and 402 retry."""
from __future__ import annotations
from dataclasses import dataclass
import structlog
from .client import Pay2StoreClient, StorageReceipt
from .parsec_signer import ParsecSigner
from .mind_layer import ArchivalDecision
from .exceptions import ArchivalExecutionError

log = structlog.get_logger(__name__)

@dataclass(frozen=True, slots=True)
class ArchivalReceipt:
    ar_txid: str
    sha256: str
    size_bytes: int
    cost_usd: float
    payment_txid: str
    rail: str          # "usdc-base" | "eth" | "algo"
    tier: str
    tags: dict[str, str]

async def execute_archival(
    decision: ArchivalDecision,
    payload: bytes,
    *,
    client: Pay2StoreClient,
    signer: ParsecSigner,
    agent_id: str,
) -> ArchivalReceipt:
    if not decision.archive:
        raise ArchivalExecutionError("decision.archive is False")
    try:
        receipt: StorageReceipt = await client.store(
            data=payload, tags=decision.tags, signer=signer,
            max_cost_usd=decision.max_cost_usd, agent_id=agent_id,
        )
    except Exception as e:
        log.error("hands.store_failed", agent_id=agent_id, err=str(e))

```

```
        raise ArchivalExecutionError(str(e)) from e

log.info("hands.archived", agent_id=agent_id, ar_txid=receipt.ar_txid)
return ArchivalReceipt(
    ar_txid=receipt.ar_txid, sha256=receipt.sha256, size_bytes=receipt.size_bytes,
    cost_usd=receipt.cost_usd, payment_txid=receipt.payment_txid,
    rail=receipt.rail, tier=decision.tier, tags=decision.tags,
)
```

Section 3 — Policy engine (`policy.py`)

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Composable policy engine for archival decisions."""
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from pathlib import Path
import time
import yaml
import structlog
from .budget import BudgetTracker
from .exceptions import PolicyConfigError

log = structlog.get_logger(__name__)

@dataclass(frozen=True, slots=True)
class PolicyDecision:
    allow: bool
    reason: str
    additional_tags: dict[str, str] = field(default_factory=dict)

@dataclass
class PolicyContext:
    agent_id: str
    size_bytes: int
    content_type: str
    payload_sha256: str
    importance: float
    tier: str

class PolicyRule(ABC):
    name: str = "AbstractRule"
    @abstractmethod
    async def check(self, ctx: PolicyContext) -> PolicyDecision: ...

class BudgetCapRule(PolicyRule):
    name = "BudgetCapRule"
    def __init__(self, budget: BudgetTracker): self.budget = budget
    async def check(self, ctx):

```

```
    ok = await self.budget.can_afford(ctx.agent_id, 0.0, tier=ctx.tier)
    return PolicyDecision(ok, "budget_ok" if ok else "budget_exhausted")
```

```
class SizeFloorRule(PolicyRule):
    name = "SizeFloorRule"
    def __init__(self, min_bytes: int = 1024): self.min_bytes = min_bytes
    async def check(self, ctx):
        ok = ctx.size_bytes >= self.min_bytes
        return PolicyDecision(ok, "size_ok" if ok else f"under_floor_{self.min_bytes}")
```

```
class SizeCeilingRule(PolicyRule):
    name = "SizeCeilingRule"
    def __init__(self, max_bytes: int = 100 * 1024): # 100 KiB free tier
        self.max_bytes = max_bytes
    async def check(self, ctx):
        ok = ctx.size_bytes <= self.max_bytes
        return PolicyDecision(ok, "size_ok" if ok else f"over_ceiling_{self.max_bytes}")
```

```
class ContentTypeWhitelist(PolicyRule):
    name = "ContentTypeWhitelist"
    def __init__(self, allowed: list[str]): self.allowed = set(allowed)
    async def check(self, ctx):
        ok = ctx.content_type in self.allowed
        return PolicyDecision(ok, f"content_type_{'ok' if ok else 'denied'}")
```

```
class DuplicateDetectionRule(PolicyRule):
    name = "DuplicateDetectionRule"
    def __init__(self, lookup): # async (sha256) -> Optional[str]
        self.lookup = lookup
    async def check(self, ctx):
        existing = await self.lookup(ctx.payload_sha256)
        if existing:
            return PolicyDecision(False, f"duplicate_of_{existing}",
                                   {"X-Duplicate-Of": existing})
        return PolicyDecision(True, "novel")
```

```
class LegalReviewRule(PolicyRule):
    """Flag PII / copyrighted content. Synchronous heuristic + tag injection."""
    name = "LegalReviewRule"
```

```

def __init__(self, scanner): # async (bytes|str) -> dict[str,bool]
    self.scanner = scanner
async def check(self, ctx):
    flags = await self.scanner(ctx.payload_sha256)
    if flags.get("requires_human_review"):
        return PolicyDecision(False, "legal_review_required",
                               {"X-Legal-Flag": "human-review"})
    return PolicyDecision(True, "legal_clear")

```

```

class RateLimitRule(PolicyRule):
    name = "RateLimitRule"
    def __init__(self, max_per_hour: int = 60):
        self.max_per_hour = max_per_hour
        self._buckets: dict[str, list[float]] = {}
    async def check(self, ctx):
        now = time.time(); cutoff = now - 3600
        bucket = [t for t in self._buckets.get(ctx.agent_id, []) if t > cutoff]
        if len(bucket) >= self.max_per_hour:
            self._buckets[ctx.agent_id] = bucket
            return PolicyDecision(False, "rate_limited")
        bucket.append(now); self._buckets[ctx.agent_id] = bucket
        return PolicyDecision(True, "rate_ok")

```

```

class ImportanceThresholdRule(PolicyRule):
    name = "ImportanceThresholdRule"
    def __init__(self, threshold: float = 0.5): self.threshold = threshold
    async def check(self, ctx):
        ok = ctx.importance >= self.threshold
        return PolicyDecision(ok, f"importance_{'pass' if ok else 'low'}")

```

```

class PolicyEngine:
    """AND-composes a list of rules. OR composition done via subclasses."""
    def __init__(self, rules: list[PolicyRule]): self.rules = rules

    async def evaluate(self, **ctx_kwargs) -> PolicyDecision:
        ctx = PolicyContext(**ctx_kwargs)
        merged_tags: dict[str, str] = {}
        for rule in self.rules:
            d = await rule.check(ctx)
            merged_tags.update(d.additional_tags)
            if not d.allow:

```

```

        log.info("policy.deny", rule=rule.name, reason=d.reason, agent_id=ct)
        return PolicyDecision(False, f"{rule.name}:{d.reason}", merged_tags)
    return PolicyDecision(True, "all_passed", merged_tags)

_RULE_REGISTRY: dict[str, type[PolicyRule]] = {
    cls.name: cls for cls in (
        SizeFloorRule, SizeCeilingRule, ContentTypeWhitelist,
        RateLimitRule, ImportanceThresholdRule)
}

def load_policy_yaml(path: Path, *, budget: BudgetTracker, **deps) -> PolicyEngine:
    spec = yaml.safe_load(Path(path).read_text())
    if not isinstance(spec, dict) or "rules" not in spec:
        raise PolicyConfigError("invalid policy yaml: missing 'rules' list")
    rules: list[PolicyRule] = [BudgetCapRule(budget)]
    for r in spec["rules"]:
        name = r.pop("rule")
        if name in _RULE_REGISTRY:
            rules.append(_RULE_REGISTRY[name](**r))
        elif name == "DuplicateDetectionRule":
            rules.append(DuplicateDetectionRule(deps["dedupe_lookup"]))
        elif name == "LegalReviewRule":
            rules.append(LegalReviewRule(deps["legal_scanner"]))
        else:
            raise PolicyConfigError(f"unknown rule: {name}")
    return PolicyEngine(rules)

```

`policy.example.yaml`:

yaml


```
version: 1
rules:
  - rule: SizeFloorRule
    min_bytes: 1024
  - rule: SizeCeilingRule
    max_bytes: 102400      # 100 KiB Turbo free tier
  - rule: ContentTypeWhitelist
    allowed: ["application/json", "text/plain", "text/markdown",
              "application/x-mindx-thought+json"]
  - rule: ImportanceThresholdRule
    threshold: 0.5
  - rule: RateLimitRule
    max_per_hour: 120
  - rule: DuplicateDetectionRule
  - rule: LegalReviewRule
```

Section 4 — Budget tracking (`budget.py`)

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Per-agent USD budget tracking with Postgres-backed atomic deduction."""
from __future__ import annotations
from dataclasses import dataclass
from datetime import date, datetime, timezone
from decimal import Decimal
import asyncpg
import structlog

log = structlog.get_logger(__name__)

SCHEMA_SQL = """
CREATE TABLE IF NOT EXISTS budget_allocations (
    agent_id      TEXT          NOT NULL,
    period        TEXT          NOT NULL,          -- 'daily'|'monthly'|'lifetime'
    period_key    TEXT          NOT NULL,          -- e.g. '2026-05-09' or '2026-05'
    tier          TEXT          NOT NULL,          -- 'cold'|'warm'|'hot'|'*'
    cap_usd       NUMERIC(18,6) NOT NULL,
    spent_usd     NUMERIC(18,6) NOT NULL DEFAULT 0,
    auto_paused   BOOLEAN      NOT NULL DEFAULT FALSE,
    version       BIGINT       NOT NULL DEFAULT 0,
    updated_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    PRIMARY KEY (agent_id, period, period_key, tier)
);
CREATE INDEX IF NOT EXISTS idx_budget_agent_period
    ON budget_allocations(agent_id, period);
"""

@dataclass(frozen=True, slots=True)
class BudgetSnapshot:
    agent_id: str
    period: str
    period_key: str
    tier: str
    cap_usd: Decimal
    spent_usd: Decimal
    auto_paused: bool
    @property
    def remaining(self) -> Decimal: return self.cap_usd - self.spent_usd
    @property

```

```
def fraction(self) -> float:
    return float(self.spent_usd / self.cap_usd) if self.cap_usd else 1.0
```

```
class BudgetTracker:
```

```
    def __init__(self, pool: asyncpg.Pool, *, alert_thresholds=(0.8, 0.95, 1.0)):
        self.pool = pool
        self.alert_thresholds = alert_thresholds
```

```
    @staticmethod
```

```
    def _keys(now: datetime) -> dict[str, str]:
        return {"daily": now.strftime("%Y-%m-%d"),
                "monthly": now.strftime("%Y-%m"),
                "lifetime": "ALL"}
```

```
    async def init_schema(self) -> None:
        async with self.pool.acquire() as c: await c.execute(SCHEMA_SQL)
```

```
    async def can_afford(self, agent_id: str, cost_usd: float, *, tier: str) -> bool:
        snap = await self._snapshot(agent_id, tier)
        for s in snap:
            if s.auto_paused: return False
            if s.spent_usd + Decimal(str(cost_usd)) > s.cap_usd: return False
        return True
```

```
    async def _snapshot(self, agent_id: str, tier: str) -> list[BudgetSnapshot]:
        keys = self._keys(datetime.now(timezone.utc))
        out: list[BudgetSnapshot] = []
        async with self.pool.acquire() as c:
            for period, key in keys.items():
                rows = await c.fetch(
                    """SELECT * FROM budget_allocations
                       WHERE agent_id=$1 AND period=$2 AND period_key=$3
                          AND tier IN ($4, '*')""",
                    agent_id, period, key, tier)
                for r in rows:
                    out.append(BudgetSnapshot(
                        agent_id=r["agent_id"], period=r["period"],
                        period_key=r["period_key"], tier=r["tier"],
                        cap_usd=r["cap_usd"], spent_usd=r["spent_usd"],
                        auto_paused=r["auto_paused"]))
        return out
```

```
    async def reserve_and_charge(self, agent_id: str, cost_usd: float, *, tier: str)
```

```

"""Atomic optimistic-concurrency deduction across all matching rows."""
keys = self._keys(datetime.now(timezone.utc))
cost = Decimal(str(cost_usd))
async with self.pool.acquire() as c, c.transaction():
    for period, key in keys.items():
        row = await c.fetchrow(
            """SELECT spent_usd, cap_usd, version, auto_paused
            FROM budget_allocations
            WHERE agent_id=$1 AND period=$2 AND period_key=$3
            AND tier IN ($4, '*')
            FOR UPDATE""",
            agent_id, period, key, tier)
        if row is None: continue
        if row["auto_paused"]: return False
        new_spent = row["spent_usd"] + cost
        if new_spent > row["cap_usd"]:
            await c.execute(
                """UPDATE budget_allocations SET auto_paused=TRUE,
                updated_at=NOW(), version=version+1
                WHERE agent_id=$1 AND period=$2 AND period_key=$3
                AND tier IN ($4, '*')""",
                agent_id, period, key, tier)
            return False
        await c.execute(
            """UPDATE budget_allocations
            SET spent_usd=$1, updated_at=NOW(), version=version+1
            WHERE agent_id=$2 AND period=$3 AND period_key=$4
            AND tier IN ($5, '*') AND version=$6""",
            new_spent, agent_id, period, key, tier, row["version"])
        fraction = float(new_spent / row["cap_usd"])
        for t in self.alert_thresholds:
            if row["spent_usd"] / row["cap_usd"] < t <= fraction:
                log.warning("budget.alert", agent_id=agent_id,
                    period=period, threshold=t)

    return True

async def adjust(self, agent_id: str, period: str, period_key: str,
    tier: str, cap_usd: float) -> None:
    async with self.pool.acquire() as c:
        await c.execute(
            """INSERT INTO budget_allocations
            (agent_id, period, period_key, tier, cap_usd)
            VALUES ($1,$2,$3,$4,$5)
            ON CONFLICT (agent_id, period, period_key, tier)

```

```
DO UPDATE SET cap_usd=EXCLUDED.cap_usd, auto_paused=FALSE,  
              updated_at=NOW(), version=budget_allocations.version  
agent_id, period, period_key, tier, Decimal(str(cap_usd)))
```

Section 5 — Memory bridge (`memory_bridge.py`)

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Bridge from archival receipts back into the mindX memory subsystems."""
from __future__ import annotations
from dataclasses import asdict
from typing import Any
import json
import structlog
from nats.aio.client import Client as NATSClient
from .hands_layer import ArchivalReceipt
from .kuzu_index import KuzuIndex
from .qdrant_index import QdrantIndex
from .meilisearch_index import MeilisearchIndex

log = structlog.get_logger(__name__)

class MindXMemoryBridge:
    def __init__(self, *, nats: NATSClient, kuzu: KuzuIndex,
                  qdrant: QdrantIndex, meili: MeilisearchIndex,
                  memory_log_writer):
        self.nats = nats; self.kuzu = kuzu; self.qdrant = qdrant
        self.meili = meili; self.memory_log = memory_log_writer

    async def commit(self, receipt: ArchivalReceipt, *, agent_id: str,
                    thought_id: str, content: str,
                    embedding: list[float]) -> None:
        # 1. JetStream publish (durable)
        js = self.nats.jetstream()
        await js.publish(
            "mindx.memory.archive_complete",
            json.dumps(**asdict(receipt), "agent_id": agent_id,
                       "thought_id": thought_id}).encode(),
            headers={"Nats-Msg-Id": receipt.ar_txid},    # idempotency
        )
        # 2. Kuzu graph insert
        await self.kuzu.link_memory_to_arweave(
            memory_id=thought_id, ar_txid=receipt.ar_txid,
            agent_id=agent_id, tier=receipt.tier,
            sha256=receipt.sha256, size=receipt.size_bytes,
            cost_usd=receipt.cost_usd)
        # 3. Qdrant vector upsert
        await self.qdrant.upsert(

```

```

        id=thought_id, vector=embedding,
        payload={"ar_txid": receipt.ar_txid, "agent_id": agent_id,
                 "tier": receipt.tier, "tags": receipt.tags,
                 "sha256": receipt.sha256})

# 4. Meilisearch full text
await self.meili.index({
    "id": thought_id, "agent_id": agent_id,
    "ar_txid": receipt.ar_txid, "content": content,
    "tier": receipt.tier, "tags": receipt.tags})

# 5. mindX append-only log
await self.memory_log.append({
    "type": "archive_committed", "thought_id": thought_id,
    "ar_txid": receipt.ar_txid, "archive_status": "permanent",
    "tier": receipt.tier, "cost_usd": receipt.cost_usd})
log.info("memory.bridge_committed", thought_id=thought_id,
        ar_txid=receipt.ar_txid)

async def retrieve_by_txid(self, ar_txid: str) -> dict[str, Any]:
    return await self.kuzu.lookup_arweave(ar_txid)

async def search_semantic(self, vector: list[float], k: int = 10):
    return await self.qdrant.search(vector, k=k)

async def search_text(self, q: str, k: int = 10):
    return await self.meili.search(q, k=k)

```

Section 6 — Parsec wallet integration (`parsec_signer.py`)

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""parsec-wallet abstraction. BANKON-internal infra: define the interface."""
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass
import hashlib, hmac, secrets, time
from eth_account import Account
from eth_account.messages import encode_typed_data
from algosdk import account as algo_account
from algosdk.transaction import PaymentTxn, SignedTransaction

USDC_BASE_MAINNET = "0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913"
USDC_BASE_CHAIN_ID = 8453

@dataclass(frozen=True, slots=True)
class EIP3009Authorization:
    from_addr: str; to_addr: str; value: int
    valid_after: int; valid_before: int
    nonce_hex: str; signature_hex: str

@dataclass(frozen=True, slots=True)
class AlgorandSignedPayment:
    txid: str; signed_txn_b64: str; sender: str
    note_b64: str; lease_hex: str

class ParsecSigner(ABC):
    @abstractmethod
    async def sign_eip3009_usdc_base(
        self, *, agent_id: str, pay_to: str, value_atoms: int,
        validity_seconds: int = 600) -> EIP3009Authorization: ...
    @abstractmethod
    async def sign_algorand_payment(
        self, *, agent_id: str, receiver: str, amount_microalgos: int,
        request_sha256: bytes, suggested_params) -> AlgorandSignedPayment: ...

class LocalParsecSigner(ParsecSigner):
    """Deterministic per-agent key derivation from a master seed.

```



```

Production deployments delegate to parsec-wallet (Ledger HW backed);
this local implementation is the fallback / test interface.
"""
def __init__(self, master_seed: bytes):
    if len(master_seed) < 32:
        raise ValueError("master_seed must be >= 32 bytes")
    self._seed = master_seed

def _derive(self, agent_id: str, chain: str) -> bytes:
    # HKDF-style derivation: HMAC-SHA512(master, "BANKON|chain|agent_id")
    info = f"BANKON|{chain}|{agent_id}".encode()
    return hmac.new(self._seed, info, hashlib.sha512).digest()[32:]

async def sign_eip3009_usdc_base(self, *, agent_id, pay_to, value_atoms,
                                validity_seconds=600):
    priv = self._derive(agent_id, "base-eth")
    acct = Account.from_key(priv)
    now = int(time.time()); nonce = secrets.token_bytes(32)
    domain = {"name": "USD Coin", "version": "2",
              "chainId": USDC_BASE_CHAIN_ID,
              "verifyingContract": USDC_BASE_MAINNET}
    types = {"TransferWithAuthorization": [
        {"name": "from", "type": "address"},
        {"name": "to", "type": "address"},
        {"name": "value", "type": "uint256"},
        {"name": "validAfter", "type": "uint256"},
        {"name": "validBefore", "type": "uint256"},
        {"name": "nonce", "type": "bytes32"}]}
    message = {"from": acct.address, "to": pay_to,
               "value": value_atoms, "validAfter": 0,
               "validBefore": now + validity_seconds, "nonce": nonce}
    signable = encode_typed_data(domain, types, message)
    signed = acct.sign_message(signable)
    return EIP3009Authorization(
        from_addr=acct.address, to_addr=pay_to, value=value_atoms,
        valid_after=0, valid_before=now + validity_seconds,
        nonce_hex="0x" + nonce.hex(),
        signature_hex=signed.signature.hex())

async def sign_algorand_payment(self, *, agent_id, receiver,
                                amount_microalgos, request_sha256,
                                suggested_params):
    priv = self._derive(agent_id, "algo")

```

```

# py-algorand-sdk expects base32 keys; convert via algosdk helpers
sk_b64 = algo_account.generate_account_from_private_key # noqa: stub
# In production parsec-wallet returns the (sk, addr) pair directly.
from algosdk import encoding
sender = encoding.encode_address(priv[:32]) # pubkey from seed
txn = PaymentTxn(
    sender=sender, sp=suggested_params, receiver=receiver,
    amt=amount_microalgos, note=b"x402:" + request_sha256,
    lease=request_sha256)
signed: SignedTransaction = txn.sign(priv)
import base64
return AlgorandSignedPayment(
    txid=txn.get_txid(),
    signed_txn_b64=base64.b64encode(
        encoding.msgpack_encode(signed).encode()).decode(),
    sender=sender, note_b64=base64.b64encode(txn.note).decode(),
    lease_hex=request_sha256.hex())

```

Section 7 — HTTP client (`client.py`)

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Async client for the pay2store HTTP service implementing x402 v2."""
from __future__ import annotations
import base64, hashlib, json
from dataclasses import dataclass
import httpx
import pybreaker
import structlog
from tenacity import retry, stop_after_attempt, wait_exponential, retry_if_exception
from .parsec_signer import ParsecSigner
from .exceptions import Pay2StoreError, PaymentRequiredError

log = structlog.get_logger(__name__)

BASE_URL_DEFAULT = "https://pay2store.agenticplace.pythai.net"

@dataclass(frozen=True, slots=True)
class QuoteResponse:
    rail: str; cost_usd: float; cost_atomic: int; pay_to: str; expires_in: int

@dataclass(frozen=True, slots=True)
class StorageReceipt:
    ar_txid: str; sha256: str; size_bytes: int; cost_usd: float
    payment_txid: str; rail: str

class Pay2StoreClient:
    def __init__(self, base_url: str = BASE_URL_DEFAULT, *,
                 timeout: float = 60.0, max_connections: int = 32):
        limits = httpx.Limits(max_connections=max_connections,
                              max_keepalive_connections=16)
        self._client = httpx.AsyncClient(
            base_url=base_url, timeout=timeout, limits=limits,
            headers={"User-Agent": "mindx-pay2store/1.0"})
        self._breaker = pybreaker.CircuitBreaker(fail_max=5, reset_timeout=30)

    async def aclose(self) -> None: await self._client.aclose()

    @retry(stop=stop_after_attempt(3),
          wait=wait_exponential(multiplier=0.5, max=4.0),

```

```

        retry=retry_if_exception_type((httpx.TransportError,)))
    async def quote(self, size_bytes: int, content_type: str,
                    rail: str = "usdc-base") -> QuoteResponse:
        r = await self._client.post("/v1/quote",
                                    json={"size_bytes": size_bytes, "content_type": content_type,
                                           "rail": rail})
        r.raise_for_status()
        d = r.json()
        return QuoteResponse(rail=d["rail"], cost_usd=d["cost_usd"],
                              cost_atomic=d["cost_atomic"],
                              pay_to=d["pay_to"], expires_in=d["expires_in"])

    async def store(self, *, data: bytes, tags: dict[str, str],
                   signer: ParsecSigner, max_cost_usd: float,
                   agent_id: str, rail: str = "usdc-base") -> StorageReceipt:
        sha = hashlib.sha256(data).hexdigest()
        tags = {**tags, "Payload-Sha256": sha}
        files = {"data": ("payload.bin", data, tags.get("Content-Type", "application/octet-stream"))}
        form = {"tags": json.dumps(tags), "rail": rail, "agent_id": agent_id}

        async def _do_post(extra_headers: dict[str, str] | None = None):
            return await self._client.post("/v1/store", data=form, files=files,
                                           headers=extra_headers or {})

        # First attempt, expect 402
        r = await self._breaker.call_async(_do_post)
        if r.status_code == 200:
            return self._parse_receipt(r)
        if r.status_code != 402:
            raise Pay2StoreError(f"unexpected status {r.status_code}: {r.text}")

        pr_b64 = r.headers.get("PAYMENT-REQUIRED") or r.headers.get("X-PAYMENT-REQUIRED")
        if not pr_b64:
            try:
                pr = r.json()
            except Exception:
                raise PaymentRequiredError("no PAYMENT-REQUIRED header")
        else:
            pr = json.loads(base64.b64decode(pr_b64))

        chosen = self._select_payment_option(pr["accepts"], max_cost_usd, rail)
        sig_header = await self._sign_payment(chosen, signer, agent_id, sha)

        r2 = await self._breaker.call_async(_do_post,
                                           {"PAYMENT-SIGNATURE": sig_header})
        if r2.status_code != 200:
            raise Pay2StoreError(f"settlement failed {r2.status_code}: {r2.text}")

```

```

        return self._parse_receipt(r2)

def _select_payment_option(self, accepts: list[dict], max_cost_usd: float,
                           preferred_rail: str) -> dict:
    rail_to_caip = {"usdc-base": "eip155:8453", "eth-mainnet": "eip155:1",
                    "algo": "algorand:mainnet"}
    target = rail_to_caip.get(preferred_rail)
    for a in accepts:
        if a["network"] == target: return a
    return accepts[0]

async def _sign_payment(self, option: dict, signer: ParsecSigner,
                        agent_id: str, request_sha: str) -> str:
    if option["network"].startswith("eip155:"):
        auth = await signer.sign_eip3009_usdc_base(
            agent_id=agent_id, pay_to=option["payTo"],
            value_atoms=int(option["amount"]))
        payload = {"x402Version": 2, "scheme": "exact",
                   "network": option["network"],
                   "payload": {"signature": auth.signature_hex,
                              "authorization": {
                                  "from": auth.from_addr, "to": auth.to_addr,
                                  "value": str(auth.value),
                                  "validAfter": str(auth.valid_after),
                                  "validBefore": str(auth.valid_before),
                                  "nonce": auth.nonce_hex}}}
        return base64.b64encode(json.dumps(payload).encode()).decode()
    if option["network"].startswith("algorand:"):
        from algosdk.v2client import algod
        ac = algod.AlgodClient("", "https://mainnet-api.algonode.cloud")
        sp = ac.suggested_params()
        signed = await signer.sign_algorand_payment(
            agent_id=agent_id, receiver=option["payTo"],
            amount_microalgos=int(option["amount"]),
            request_sha256=bytes.fromhex(request_sha),
            suggested_params=sp)
        payload = {"x402Version": 2, "scheme": "exact",
                   "network": option["network"],
                   "payload": {"signedTxn": signed.signed_txn_b64,
                              "txid": signed.txid,
                              "lease": signed.lease_hex}}
        return base64.b64encode(json.dumps(payload).encode()).decode()
    raise Pay2StoreError(f"unsupported network {option['network']}")

```

```
def _parse_receipt(self, r: httpx.Response) -> StorageReceipt:
    d = r.json()
    return StorageReceipt(
        ar_txid=d["ar_txid"], sha256=d["sha256"],
        size_bytes=d["size_bytes"], cost_usd=d["cost_usd"],
        payment_txid=d["payment_txid"], rail=d["rail"])

async def verify(self, ar_txid: str) -> dict:
    r = await self._client.get(f"/v1/verify/{ar_txid}")
    r.raise_for_status()
    return r.json()
```

Section 8 — Verification + on-chain registration (`verification.py`)

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Post-upload verification + BONAFIDE on-chain registration."""
from __future__ import annotations
from dataclasses import dataclass
import hashlib
from typing import Sequence
import httpx
import structlog
from web3 import AsyncWeb3
from web3.providers.async_rpc import AsyncHTTPProvider

log = structlog.get_logger(__name__)

ARWEAVE_GATEWAY = "https://arweave.net"
ARWEAVE_GRAPHQL = "https://arweave.net/graphql"

# BONAFIDE is BANKON-internal. The interface below MUST be deployed via Foundry
# (`forge create`) to a known address before production use. Replace the
# placeholder constant with the deployed address.
BONAFIDE_REGISTRY_MAINNET = "0x000000000000000000000000000000000000B00F" # placeholder
BONAFIDE_ABI = [
    {"name": "register", "type": "function", "stateMutability": "nonpayable",
     "inputs": [{"name": "key", "type": "bytes32"}, {"name": "arweaveTx", "type": "string"}],
     "outputs": []},
    {"name": "lookup", "type": "function", "stateMutability": "view",
     "inputs": [{"name": "key", "type": "bytes32"}],
     "outputs": [{"name": "arweaveTx", "type": "string"}, {"name": "timestamp", "type": "uint256"}]}

@dataclass(frozen=True, slots=True)
class VerificationResult:
    ar_txid: str
    confirmed: bool
    confirmations: int
    sha256_match: bool
    tags_match: bool
    bundled_in: str | None

class VerificationService:

```

```

def __init__(self, *, w3: AsyncWeb3, eth_signer_key: str,
              registry_address: str = BONAFIDE_REGISTRY_MAINNET,
              finality_depth: int = 15):
    self.w3 = w3
    self.account = self.w3.eth.account.from_key(eth_signer_key)
    self.registry = self.w3.eth.contract(address=registry_address, abi=BONAFIDE_
    self.finality_depth = finality_depth

async def verify_arweave(self, ar_txid: str, expected_sha256: str,
                        expected_tags: dict[str, str]) -> VerificationResult:
    async with httpx.AsyncClient(timeout=15.0) as c:
        # status
        r_status = await c.get(f"{ARWEAVE_GATEWAY}/tx/{ar_txid}/status")
        confirmations = 0; bundled_in = None
        if r_status.status_code == 200:
            confirmations = r_status.json().get("number_of_confirmations", 0)
        elif r_status.status_code == 404:
            # likely an ANS-104 bundled data item; resolve via GraphQL
            gql = await c.post(ARWEAVE_GRAPHQL, json={"query":
                'query($id:ID!){transaction(id:$id){bundledIn{id} tags{name value}
                "variables": {"id": ar_txid}})
            node = gql.json()["data"]["transaction"]
            if node and node.get("bundledIn"):
                bundled_in = node["bundledIn"]["id"]
                rs2 = await c.get(f"{ARWEAVE_GATEWAY}/tx/{bundled_in}/status")
                if rs2.status_code == 200:
                    confirmations = rs2.json().get("number_of_confirmations", 0)

        # data + sha256
        r_data = await c.get(f"{ARWEAVE_GATEWAY}/{ar_txid}")
        r_data.raise_for_status()
        sha = hashlib.sha256(r_data.content).hexdigest()
        sha_match = (sha == expected_sha256)

        # tags
        gql2 = await c.post(ARWEAVE_GRAPHQL, json={"query":
            'query($id:ID!){transaction(id:$id){tags{name value}}}',
            "variables": {"id": ar_txid}})
        node = gql2.json()["data"]["transaction"]
        actual = {t["name"]: t["value"] for t in (node["tags"] if node else [])}
        tags_match = all(actual.get(k) == v for k, v in expected_tags.items())

    return VerificationResult(ar_txid, confirmations >= self.finality_depth,
                             confirmations, sha_match, tags_match, bundled_in)

async def register_in_bonafide(self, ar_txid: str, *, key_seed: bytes) -> str:

```



```

key = hashlib.sha256(key_seed).digest()
tx = await self.registry.functions.register(key, ar_txid).build_transaction(
    "from": self.account.address,
    "nonce": await self.w3.eth.get_transaction_count(self.account.address),
    "chainId": 1,
    "maxFeePerGas": await self.w3.eth.gas_price * 2,
    "maxPriorityFeePerGas": self.w3.to_wei(1, "gwei"),
    "gas": 120_000,
})
signed = self.account.sign_transaction(tx)
h = await self.w3.eth.send_raw_transaction(signed.raw_transaction)
log.info("bonafide.registered", ar_txid=ar_txid, eth_tx=h.hex())
return h.hex()

async def daily_merkle_commit(self, ar_txids: Sequence[str]) -> str:
    """Commit a Merkle root of the day's archives in a single tx."""
    leaves = [hashlib.sha256(t.encode()).digest() for t in ar_txids]
    while len(leaves) > 1:
        if len(leaves) % 2: leaves.append(leaves[-1])
        leaves = [hashlib.sha256(leaves[i] + leaves[i+1]).digest()
                  for i in range(0, len(leaves), 2)]
    root = leaves[0]
    return await self.register_in_bonafide(root.hex(), key_seed=b"daily-root")

```

Section 9 — Tag taxonomy (`tag_taxonomy.py`)

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Canonical mindX tag schemas. Pydantic v2 for validation."""
from __future__ import annotations
from typing import Literal
from pydantic import BaseModel, Field, field_validator

CanonicalKind = Literal[
    "mindx.thought", "mindx.memory.consolidation", "mindx.knowledge.fact",
    "mindx.training.checkpoint", "mindx.dialogue.transcript",
    "mindx.constitution.amendment", "mindx.attestation.identity"]

class BaseTags(BaseModel):
    app_name: str = Field(default="mindX", alias="App-Name")
    kind: CanonicalKind = Field(alias="Kind")
    agent_id: str = Field(alias="Agent-Id")
    payload_sha256: str = Field(alias="Payload-Sha256")
    content_type: str = Field(alias="Content-Type")
    tier: Literal["cold", "warm", "hot"] = Field(alias="Tier")
    schema_version: str = Field(default="1.0.0", alias="Schema-Version")

    @field_validator("payload_sha256")
    @classmethod
    def _hex64(cls, v: str) -> str:
        if len(v) != 64 or any(c not in "0123456789abcdef" for c in v.lower()):
            raise ValueError("Payload-Sha256 must be 64 hex chars")
        return v.lower()

    model_config = {"populate_by_name": True}

class ThoughtTags(BaseTags):
    kind: Literal["mindx.thought"] = Field(default="mindx.thought", alias="Kind")
    thought_id: str = Field(alias="Thought-Id")
    parent_thought_id: str | None = Field(default=None, alias="Parent-Thought-Id")

class ConstitutionAmendmentTags(BaseTags):
    kind: Literal["mindx.constitution.amendment"] = Field(
        default="mindx.constitution.amendment", alias="Kind")
    amendment_id: str = Field(alias="Amendment-Id")
    daio_vote_root: str = Field(alias="DAIO-Vote-Root")
    effective_block: int = Field(alias="Effective-Block")

```

```
class TrainingCheckpointTags(BaseTags):
    kind: Literal["mindx.training.checkpoint"] = Field(
        default="mindx.training.checkpoint", alias="Kind")
    model_id: str = Field(alias="Model-Id")
    step: int = Field(alias="Step")
    base_checkpoint_tx: str | None = Field(default=None, alias="Base-Checkpoint-Tx")
```

Section 10 — NATS JetStream consumer (`nats_consumer.py`)

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Pull consumer over mindx.memory.archive_request – idempotent, DLQ-aware."""
from __future__ import annotations
import asyncio, json
import nats
from nats.errors import TimeoutError as NATSTimeout
import structlog
from .service import MindXPay2StoreService

log = structlog.get_logger(__name__)

STREAM = "MINDX_MEMORY"
SUBJECT_REQUEST = "mindx.memory.archive_request"
SUBJECT_DLQ = "mindx.memory.archive_dlq"

class ArchivalEventConsumer:
    def __init__(self, *, nats_url: str, service: MindXPay2StoreService,
                 batch: int = 8, durable: str = "pay2store-worker"):
        self.nats_url = nats_url; self.service = service
        self.batch = batch; self.durable = durable
        self._stop = asyncio.Event()

    async def run(self) -> None:
        nc = await nats.connect(self.nats_url)
        js = nc.jetstream()
        await js.add_stream(name=STREAM,
                           subjects=[SUBJECT_REQUEST, SUBJECT_DLQ,
                                    "mindx.memory.archive_complete"])
        psub = await js.pull_subscribe(SUBJECT_REQUEST,
                                       durable=self.durable, stream=STREAM)
        log.info("consumer.start", durable=self.durable)
        while not self._stop.is_set():
            try:
                msgs = await psub.fetch(self.batch, timeout=5)
            except NATSTimeout:
                continue
            for msg in msgs:
                try:
                    await msg.in_progress()
                    payload = json.loads(msg.data)
                    receipt = await self.service.archive_from_event(payload)

```

```

        await msg.ack()
        log.info("consumer.ack", ar_txid=receipt.ar_txid)
    except Exception as e:
        headers = msg.headers or {}
        deliv = int(headers.get("Nats-Delivery-Count", "1"))
        if deliv >= 5:
            await js.publish(SUBJECT_DLQ, msg.data,
                            headers={"Original-Subject": SUBJECT_REQUEST,
                                     "Error": str(e)[:256]})
            await msg.ack()
            log.error("consumer.dlq", err=str(e))
        else:
            await msg.nak(delay=2 ** deliv)
            log.warning("consumer.nak", attempt=deliv, err=str(e))

def stop(self) -> None: self._stop.set()

```

Section 11 — Internal FastAPI (`api.py`)

python

```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""mindX-internal HTTP API for service control."""
from __future__ import annotations
from fastapi import APIRouter, HTTPException, Depends
from pydantic import BaseModel
from .service import MindXPay2StoreService

router = APIRouter(prefix="/pay2store", tags=["pay2store"])

def get_service() -> MindXPay2StoreService: ...    # bound at app startup


class ArchiveRequest(BaseModel):
    agent_id: str; thought_id: str
    content: str; content_type: str = "application/x-mindx-thought+json"
    importance: float = 0.5; novelty: float = 0.5; reach: float = 0.5
    permanence_required: bool = False


class BudgetUpdate(BaseModel):
    period: str; period_key: str; tier: str; cap_usd: float


@router.get("/health")
async def health(svc: MindXPay2StoreService = Depends(get_service)):
    return {"status": "ok", "version": "1.0.0", **(await svc.health())}


@router.post("/archive")
async def archive(req: ArchiveRequest,
                 svc: MindXPay2StoreService = Depends(get_service)):
    try:
        receipt = await svc.archive_thought(
            agent_id=req.agent_id, thought_id=req.thought_id,
            content=req.content.encode(), content_type=req.content_type,
            importance=req.importance, novelty=req.novelty,
            reach=req.reach, permanence_required=req.permanence_required)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
    return receipt.__dict__

```

```

@router.get("/budget/{agent_id}")
async def budget_show(agent_id: str,
                       svc: MindXPay2StoreService = Depends(get_service)):
    return await svc.budget_snapshot(agent_id)

@router.put("/budget/{agent_id}")
async def budget_set(agent_id: str, body: BudgetUpdate,
                     svc: MindXPay2StoreService = Depends(get_service)):
    await svc.adjust_budget(agent_id, **body.model_dump())
    return {"ok": True}

@router.get("/receipts")
async def receipts_list(limit: int = 50,
                        svc: MindXPay2StoreService = Depends(get_service)):
    return await svc.list_receipts(limit=limit)

@router.get("/receipts/{tx_id}")
async def receipt_get(tx_id: str,
                      svc: MindXPay2StoreService = Depends(get_service)):
    r = await svc.get_receipt(tx_id)
    if not r: raise HTTPException(404, "not found")
    return r

@router.post("/policy/test")
async def policy_test(req: ArchiveRequest,
                      svc: MindXPay2StoreService = Depends(get_service)):
    return await svc.dry_run_policy(req.model_dump())

```

Section 12 — Metrics (`metrics.py`)

python

```
# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""Prometheus metrics."""
from prometheus_client import Counter, Gauge, Histogram

archives_total = Counter("mindx_pay2store_archives_total",
    "Total archive operations.", ["agent_id", "tier", "status"])
bytes_total = Counter("mindx_pay2store_bytes_total",
    "Bytes archived.", ["agent_id", "tier"])
cost_usd_total = Counter("mindx_pay2store_cost_usd_total",
    "USD cost of archives.", ["agent_id", "currency"])
latency_seconds = Histogram("mindx_pay2store_latency_seconds",
    "Latency of operations.", ["operation"],
    buckets=(0.05, 0.1, 0.25, 0.5, 1, 2, 5, 10, 30, 60))
budget_remaining_usd = Gauge("mindx_pay2store_budget_remaining_usd",
    "Remaining budget.", ["agent_id", "period"])
policy_decisions_total = Counter("mindx_pay2store_policy_decisions_total",
    "Policy rule outcomes.", ["rule", "decision"])
```

Section 13 — CLI (`cli.py`)

```
python
```



```

# (c) 2026 BANKON – all rights reserved
# Licensed under the Apache License, Version 2.0
"""python -m mindx_pay2store ..."""
from __future__ import annotations
import asyncio, sys
from pathlib import Path
import typer
from .config import Settings
from .service import build_service

app = typer.Typer(no_args_is_help=True, add_completion=False)

@app.command()
def archive(file: Path, agent: str = typer.Option(...),
            tier: str = typer.Option("cold")):
    asyncio.run(_archive(file, agent, tier))

async def _archive(file: Path, agent: str, tier: str) -> None:
    svc = await build_service(Settings())
    r = await svc.archive_thought(
        agent_id=agent, thought_id=file.stem, content=file.read_bytes(),
        content_type="application/octet-stream",
        importance=1.0, novelty=1.0, reach=0.5, permanence_required=True)
    typer.echo(f"ar://{r.ar_txid}  ${r.cost_usd:.6f}  {r.size_bytes}B  rail={r.rail}")

@app.command()
def verify(txid: str):
    asyncio.run(_verify(txid))

async def _verify(txid: str) -> None:
    svc = await build_service(Settings())
    typer.echo(repr(await svc.verify(txid)))

budget_app = typer.Typer(); app.add_typer(budget_app, name="budget")

@budget_app.command("show")
def budget_show(agent_id: str):
    async def _go():

```

```

        svc = await build_service(Settings())
        typer.echo(repr(await svc.budget_snapshot(agent_id)))
    asyncio.run(_go())

@budget_app.command("set")
def budget_set(agent_id: str, daily_usd: float = typer.Option(...)):
    async def _go():
        svc = await build_service(Settings())
        from datetime import date
        await svc.adjust_budget(agent_id, period="daily",
                                period_key=date.today().isoformat(), tier="*", cap_usd=daily_usd)
        typer.echo("ok")
    asyncio.run(_go())

@app.command()
def query(q: str, k: int = 10):
    async def _go():
        svc = await build_service(Settings())
        for hit in await svc.search_semantic(q, k=k):
            typer.echo(f"{hit['ar_txid']} {hit['score']:.3f} {hit['payload'].get('')}")
    asyncio.run(_go())

@app.command("policy-validate")
def policy_validate(yaml_path: Path):
    from .policy import load_policy_yaml
    try:
        load_policy_yaml(yaml_path, budget=None) # type: ignore[arg-type]
        typer.echo("ok")
    except Exception as e:
        typer.echo(f"INVALID: {e}", err=True); sys.exit(1)

if __name__ == "__main__":
    app()

```

Section 14 — Testing (excerpt)

`tests/test_policy.py` (representative; full suite mirrors structure for service, budget, memory_bridge, integration):

python

```

# (c) 2026 BANKON – all rights reserved
import pytest
from hypothesis import given, strategies as st
from mindx_pay2store.policy import (PolicyEngine, SizeFloorRule,
                                     SizeCeilingRule, ContentTypeWhitelist, ImportanceThresholdRule)

@pytest.mark.asyncio
async def test_floor_rejects_small_payloads():
    eng = PolicyEngine([SizeFloorRule(1024)])
    d = await eng.evaluate(agent_id="a", size_bytes=10,
                           content_type="text/plain", payload_sha256="0"*64,
                           importance=1.0, tier="cold")
    assert not d.allow and "SizeFloorRule" in d.reason

@given(st.integers(min_value=0, max_value=10**8))
@pytest.mark.asyncio
async def test_ceiling_property(size):
    eng = PolicyEngine([SizeCeilingRule(102_400)])
    d = await eng.evaluate(agent_id="a", size_bytes=size,
                           content_type="text/plain", payload_sha256="0"*64,
                           importance=1.0, tier="cold")
    assert d.allow == (size <= 102_400)

```

tests/test_integration.py uses respx to mock pay2store.agenticplace.pythai.net returning a 402 then 200, asserts the EIP-3009 typed-data signature recovers the agent's derived address, and uses testcontainers[postgres,nats,qdrant] to spin real backends. A locust file (tests/locustfile.py) drives the /pay2store/archive endpoint at sustained 50 RPS to validate the budget tracker's optimistic-concurrency path.

Section 15 — Deployment

Containerfile (Podman):

```
dockerfile
```

```
# (c) 2026 BANKON – all rights reserved
FROM docker.io/library/python:3.12-slim-bookworm
ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /opt/mindx_pay2store
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev curl ca-certificates && rm -rf /var/lib/apt/lists/*
COPY pyproject.toml ./
RUN pip install --upgrade pip && pip install .
COPY mindx_pay2store/ ./mindx_pay2store/
USER 65534:65534
EXPOSE 8088
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD curl -fsS http://localhost:8088/pay2store/health || exit 1
CMD ["uvicorn", "mindx_pay2store.app:app", "--host", "0.0.0.0", "--port", "8088"]
```

`mindx-pay2store.service` (systemd):

```
ini

[Unit]
Description=mindX pay2store service
After=network-online.target postgresql.service nats.service
Requires=network-online.target

[Service]
Type=exec
User=mindx
EnvironmentFile=/etc/mindx/pay2store.env
ExecStart=/usr/bin/podman run --rm --name mindx-pay2store \
    --env-file /etc/mindx/pay2store.env -p 8088:8088 \
    localhost/mindx-pay2store:1.0
Restart=on-failure
RestartSec=5s
NoNewPrivileges=true
ProtectSystem=strict

[Install]
WantedBy=multi-user.target
```

OpenBSD `(vmm)/(vmd)` deployment uses an Alpine guest with the same Containerfile run under Podman; secrets are sealed via HashiCorp Vault's KV-v2 engine, fetched at boot by `(vault agent)` into `(/run/mindx/pay2store.env)`. Environment schema is a Pydantic v2 `(Settings)` class in `(config.py)` covering: `(MINDX_PAY2STORE_URL)`, `(MINDX_PG_DSN)`, `(MINDX_NATS_URL)`,

`MINDX_QDRANT_URL`, `MINDX_MEILI_URL`, `MINDX_KUZU_PATH`, `BANKON_PARSEC_SEED` (32+ bytes, never logged), `BANKON_BONAFIDE_REGISTRY`, `BANKON_ETH_RPC`, `BANKON_ETH_SIGNER_KEY`. Health probes hit `/pay2store/health`; readiness additionally pings the upstream pay2store base URL. A Grafana dashboard JSON (omitted for brevity but generated from `metrics.py` series) plots archives/sec, p50/p95/p99 latency, budget exhaustion percentage per agent, and policy denial rate per rule.

Section 16 — AgenticPlace manifest integration

yaml

```
# agent.yaml (AgenticPlace manifest)
schema: agenticplace/manifest@v2
agent: epimenides
capabilities:
  - tool.mindx.pay2store
    version: "1.0"
    handler: mindx_pay2store.service:MindXPay2StoreService.archive_thought
    permissions: [archive.cold, archive.warm, budget.read, receipts.read]
    budgets:
      daily_usd: 5.00
      monthly_usd: 100.00
    policy_overrides:
      - rule: SizeFloorRule
        min_bytes: 1024
      - rule: ImportanceThresholdRule
        threshold: 0.65
```

The mindX runtime reads this manifest at agent boot, instantiates a `MindXPay2StoreService` with the per-agent overrides merged on top of the system policy YAML, and registers the resulting bound method as a tool the agent can call.

Section 17 — Worked end-to-end example

The agent `epimenides` (named for the Cretan philosopher whose paradox foreshadowed Gödel) emits a 250-line philosophical reflection titled *"Emergence in Agent Collectives"* — about 14 KiB of UTF-8. The Soul layer's `infer_archival_intent` runs the heuristic regex pass, finds the phrase *"let it be known to the collective record"* (matches `PERMANENCE_MARKERS`), and asks mindX to self-reflect. mindX returns `{permanence_required: true, urgency: "soon", suggested_tier: "warm", confidence: 0.93}`. An `ArchivalIntent` forms.

The Mind layer scores: novelty 0.92 (low cosine similarity against existing Qdrant memory), reach 0.85 (high projected fan-out via the constitutional council channel), importance 0.88.

`_stoic_score = 0.35·0.92 + 0.25·0.85 + 0.30·0.88 + 0.10·0.93 = 0.894`). With `permanence_required=True` and `score ≥ 0.85`, tier resolves to `hot` (estimated \$0.020). Policy: `SizeFloorRule(1024)` passes (`14336 ≥ 1024`); `SizeCeilingRule(102400)` passes; `ContentTypeWhitelist` passes (`application/x-mindx-thought+json`); `ImportanceThresholdRule(0.65)` passes; `RateLimitRule` passes; `DuplicateDetectionRule` consults Qdrant by SHA-256 — no hit. `BudgetCapRule` confirms epimenides has \$4.93 remaining of today's \$5.00 cap. Decision: `archive=true`, `tier=hot`, `max_cost_usd=$0.025`.

Hands layer calls `Pay2StoreClient.store(...)`. First POST returns 402 with `PAYMENT-REQUIRED` (base64 JSON) listing two `accepts`: `eip155:8453` (USDC, amount 20000 = \$0.020) and `algorand:mainnet` (ALGO fallback). The client picks Base. `LocalParsecSigner.sign_eip3009_usdc_base` derives the agent's Base key via `HMAC-SHA512(master_seed, "BANKON|base-eth|epimenides")[:32]`, composes the EIP-712 typed data with `domain.name="USD Coin"`, `version="2"`, `chainId=8453`, `verifyingContract=0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913`, signs `TransferWithAuthorization`, and returns a 65-byte signature with `validBefore = now + 600` and a fresh 32-byte nonce. The client base64-encodes the `PaymentPayload`, retries the POST with header `PAYMENT-SIGNATURE: ...`. The pay2store facilitator verifies (recovers signer == from, checks `authorizationState`, `balanceOf`), settles via on-chain `transferWithAuthorization`, uploads via `@ardrive/turbo-sdk` v1.40.2, returns 200 with `{ar_txid: "Hk8j...", sha256, size_bytes: 14336, cost_usd: 0.0198, payment_txid: "0x8f3d...", rail: "usdc-base"}`. Total wall time: ~7 s.

`VerificationService.verify_arweave` polls `arweave.net/tx/Hk8j.../status`, gets a 404 (bundled), GraphQL-resolves `bundledIn.id`, polls that L1 tx until 15 confirmations (~7 minutes on Arweave), confirms the SHA-256 of the data fetched at `arweave.net/Hk8j...` matches, and confirms the tag set. `register_in_bonafide` builds an Ethereum mainnet tx calling `BONAFIDE.register(keccak256("epimenides|<thought_id>"), "Hk8j...")`, broadcasts, and logs the L1 tx hash. The daily Merkle commit at 00:00 UTC will batch this with all other `hot`/`warm` archives of the day.

`MindXMemoryBridge.commit` then: (1) publishes to NATS subject `mindx.memory.archive_complete` with `Nats-Msg-Id: Hk8j...` for idempotency; (2) inserts in Kuzu `CREATE (m:Memory {id:'<thought_id>', agent_id:'epimenides'})-[:ARCHIVED_TO]->(a:ArweaveTx {txid:'Hk8j...', tier:'hot', cost_usd:0.0198, ts:1746800000})`; (3) embeds the reflection text via mindX's embedding service and upserts into Qdrant collection `mindx_archives` with payload pointing at the Arweave tx; (4) indexes the full text into Meilisearch `mindx_archives` index; (5) appends to epimenides's append-only log `{type:"archive_committed", thought_id, ar_txid:"Hk8j...", archive_status:"permanent", tier:"hot"}`.

Six months later the agent `heraclitus` is reasoning about emergent properties of multi-agent fleets. It calls `MindXMemoryBridge.search_semantic(embedding("emergence in collectives"), k=10)`. Qdrant returns epimenides's reflection at score 0.91 with payload `{ar_txid:"Hk8j...", tier:"hot"}`. Heraclitus resolves the content via `GET https://arweave.net/Hk8j...`, verifies the SHA-256 against the tag, and incorporates the reflection. The Kuzu graph traversal `MATCH (m:Memory)-[:ARCHIVED_TO]->(a:ArweaveTx) WHERE a.txid='Hk8j...' RETURN m.agent_id` confirms provenance: `epimenides`. The cycle closes — a thought made permanent in May was retrieved, verified, and reused in November, having survived every cache eviction along the way precisely because the Mind layer judged it worth the two cents.

Conclusion

`mindx_pay2store` reduces the question *"should this thought be made permanent?"* to a deterministic, auditable pipeline of three cognitive layers, one policy engine, one budget tracker, four persistence systems, and one ~7-second HTTP exchange against a service that already exists. The cypherpunk2048 flat layout keeps the module readable; Pydantic v2 keeps the data contracts honest; the `pybreaker`/`tenacity` pair keeps the upstream service degradation contained; and the BONAFIDE registry plus daily Merkle commit converts an Arweave txid into a verifiable on-chain BANKON identity claim. The interface boundaries are clean enough that the JS-only Turbo SDK constraint never leaks into Python-land — `pay2store` shells out, `mindx_pay2store` only speaks x402 v2 over HTTP. With the policy YAML and AgenticPlace manifest in place, an agent like `epimenides` can autonomously decide, fund, and execute its own archival without human intervention while remaining inside the budgets and rate limits the operator has set. The next operational milestones are: deploy BONAFIDE via Foundry to a fixed mainnet address; verify the multicodec prefix bytes for `arweave-ns` against `@ensdomains/content-hash` v3+ before enabling ENS contenthash updates; and pin the integration test suite at every release against testcontainers Postgres-17, NATS 2.11, and Qdrant 1.13 to guard the optimistic-concurrency path under load.